# /system

Override the agent system prompt (/system | /system help)

## Overview

`/system` sets a session-scoped override of the system prompt. The override replaces the assembled system prompt for subsequent turns in this session only; it is not written to any config file and does not survive a restart or an agent switch.

Use it to inject project-specific or task-specific instructions on top of (in place of) the current agent's prompt - for example a stricter output contract for a one-off review, without editing any prompt file.

## Syntax

```
/system                 # show the current override (if any)
/system <prompt>        # set a session-scoped system prompt override
/system help            # usage help
```

## Options / Subcommands

| Form | Description |
|---|---|
| `/system` | Show the current system-prompt override for the session |
| `/system <prompt>` | Set the override; the text is logged with its length |
| `/system help` | Print usage help |

## Examples

Show whether an override is active:

```
/system
```

Set a strict output contract for this session:

```
/system Answer with a numbered list only. No prose. Cite file paths for every claim.
```

Set a project-conventions override:

```
/system Follow AGENTS.md strictly. Run cargo fmt after every Rust edit. No unwrap() in user-facin
```

Inspect the usage help:

```
/system help
```

Clear the override by switching agents (there is no dedicated clear form):

```
/agent coder
```

## Output

- Setting an override logs a line with the character count: `System prompt set (N chars)`.
- The bare form reports the current override (or that none is set).
- The override is session-scoped: it is cleared on restart and when the active agent changes; `/system` after a restart shows no override.

## Related

- `/agent` - switching agents clears the override
- `/prompt` - inspect the assembled prompts for any agent
- `/mode` - lighter-weight role bias that does not replace the prompt
- `AGENTS.md` / custom agents - persistent prompt sources